

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 20

QUERY PROCESSING:

COST ESTIMATES

PROJECTION: ATTRIBUTE ELIMINATION

- Projection can be implemented by removing unwanted attributes from each tuple as query result tuples are produced.
 - ✎ May result in duplicate tuples unless a candidate key is in the projection.
- The query result tuples will be smaller in size than the original tuples.
 - ✎ The size reduction depends on the number and size of eliminated attributes.
 - ✎ Fewer write page I/Os are required to produce the query result.
- There is no additional cost for projection unless duplicate tuples need to be eliminated (i.e., `select distinct ...`).

PROJECTION: DUPLICATE ELIMINATION

```
select distinct boatId
from Reserves;
```

Reserves(sailorId, boatId, rDate)

- For duplicate elimination we can use **modified external sorting**.

Modify sort pass to **eliminate unwanted attributes** (e.g., sailorId, rDate).

- Thus, sorted runs contain **smaller tuples**.
(Size reduction depends on the number and size of eliminated attributes.)

Modify merge passes to **eliminate duplicates**.

- Thus, there are **fewer output tuples** than input tuples (i.e., tuples read).
(The difference depends on the number of duplicates.)

Cost: Sort pass: **read in the original relation** and **write out the same number of smaller tuples** ⇒ there are fewer write than read page I/Os since more (smaller) tuples can be placed on a page.

Merge passes: **read in smaller tuples** and **write out fewer tuples** in each pass ⇒ there are fewer write than read page I/Os since there are fewer tuples after duplicates are eliminated.

PROJECTION: DISCUSSION

- **Sorting is the standard method** for duplicate elimination since it handles skewed data, *and* the query result is sorted.
- If an **index** on the relation **contains all wanted attributes in its search key**, we can do an *index-only scan*.
 - Apply projection on the search-key part of the index entries; since index entries are much smaller than data file records there are fewer reads.
- If an **ordered (i.e., tree) index contains all wanted attributes as the *prefix* of the search key**, we can do even better.
 - Retrieve index entries in order using an index-only scan, discard unwanted attributes and compare the adjacent index entries to check for duplicates.

 **For an index-only scan, the index must be dense. Why?**

JOIN OPERATION

- Terminology:

r, s the relations to be joined.

n_r, n_s the number of tuples (records) in r and s , respectively.

B_r, B_s the number of pages in r and s , respectively

M the available buffer pages (i.e., main memory pages).

- All the examples *assume equi-join* on the following relations.

Relation	Customer	Depositor
Number of tuples	10,000	5,000
Number of pages	400	100

The join attribute is customerId, which is the key for Customer.

Customer(customerId, name, street, city)

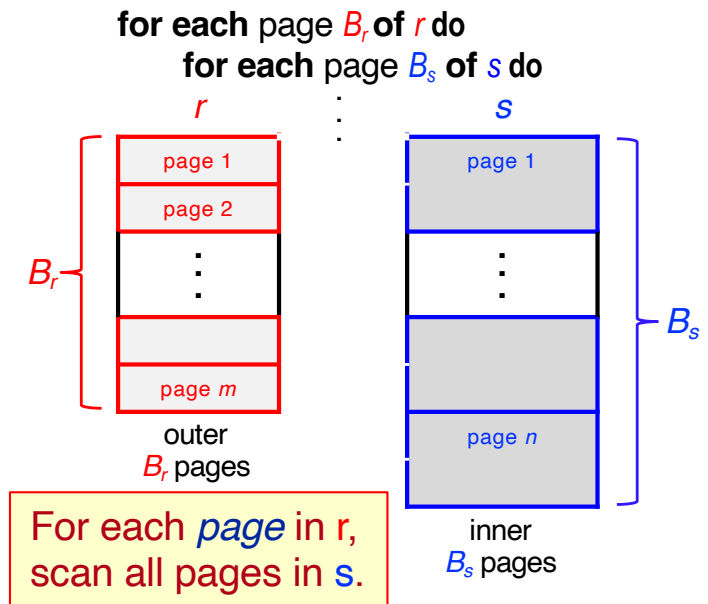
Depositor(customerId, accountNo)

BLOCK NESTED-LOOP JOIN

Requires no indexes; can be used with **any kind of join condition**.

```

for each page  $B_r$  of  $r$  do /*  $r$  outer relation */
  for each page  $B_s$  of  $s$  do /*  $s$  inner relation */
    for each tuple  $t_r$  in  $B_r$  do
      for each tuple  $t_s$  in  $B_s$  do
        if  $(t_r, t_s)$  satisfies the join condition then
          add  $(t_r, t_s)$  to the result;
  
```



Cost

Worst case: $B_r * B_s + B_r \Rightarrow$ only 1 buffer page available for each relation.

- Each page in s (the inner relation) is read only once for each page in r (the outer relation).

Best case: $B_r + B_s \Rightarrow$ when the inner relation, s , fits in the buffer.

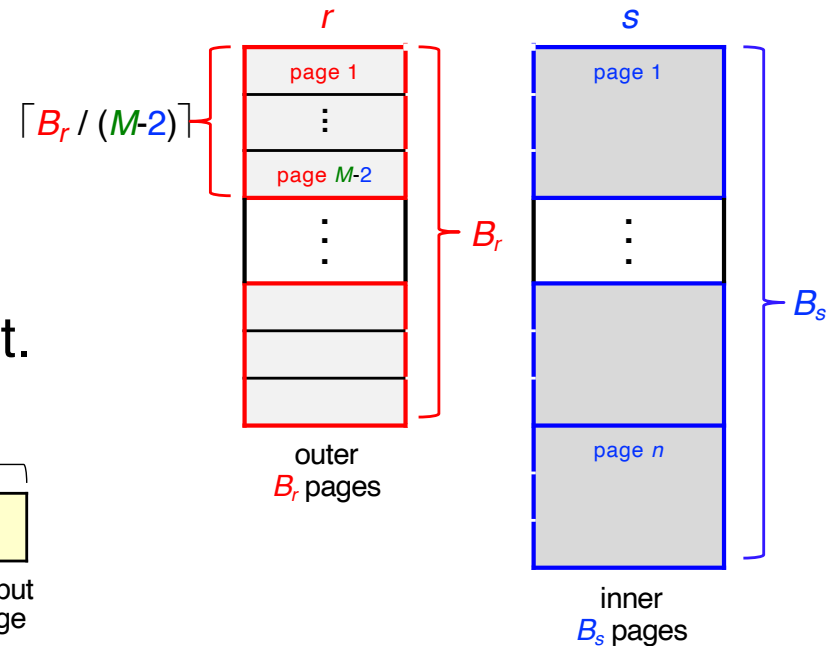
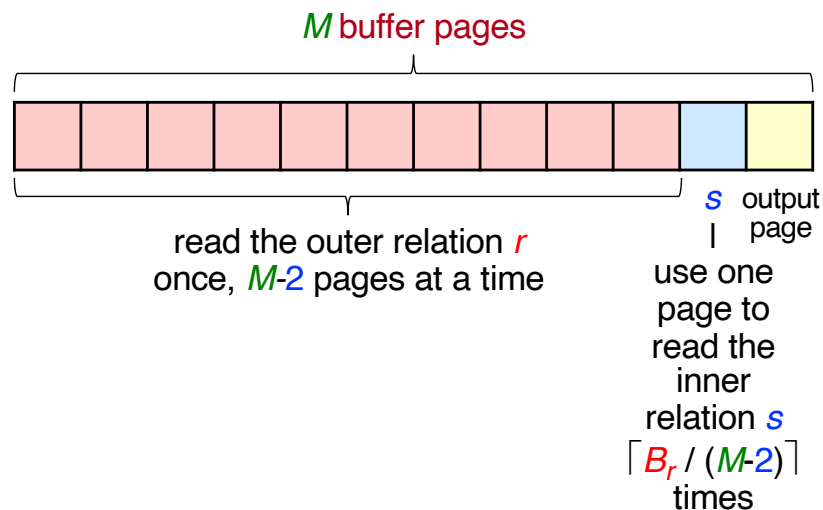
If a relation fits in the buffer, use it as the **inner relation s** .
If neither relation fits in the buffer, use the **smaller relation as the outer relation r** .

Why?

BLOCK NESTED-LOOP JOIN: OPTIMIZATIONS

1. Use $M-2$ pages as the **blocking unit** for the **outer relation r** .
(M is the buffer size in pages.)

Use the remaining **2** pages for the **inner relation s** and the output.



Cost: $\lceil B_r / (M-2) \rceil * B_s + B_r$

2. If the equi/natural-join attributes **form a key** for the inner relation, we can **stop the inner loop on the first match**.

EXAMPLE BLOCK NESTED-LOOP JOIN COST

Compute: Depositor JOIN Customer (Depositor is the outer relation)

Number of pages of $B_{\text{Depositor}} = 100$, $B_{\text{Customer}} = 400$

If neither relation fits in the buffer, use the smaller relation as the outer relation r .

Worst case cost ($B_r * B_s + B_r$)

What is the cost if we use
Customer as the outer relation?

$$100 * 400 + 100 = \underline{40,100} \text{ page I/Os}$$

Minimum buffer pages needed to achieve this cost?

Best case cost ($B_r + B_s$)

$$100 + 400 = \underline{500} \text{ page I/Os}$$

How many buffer pages are needed to achieve this cost?

Optimization case with 52 buffer pages ($\lceil B_r / (M-2) \rceil * B_s + B_r$)

$$\lceil 100 / (52-2) \rceil * 400 + 100 = 2 * 400 + 100 = \underline{900} \text{ page I/Os}$$

INDEXED NESTED-LOOP JOIN

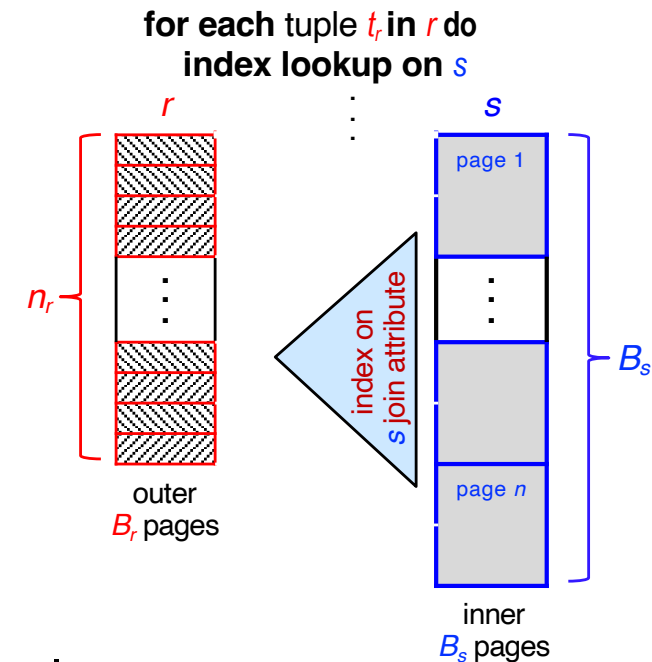
- Index lookups can replace file scans if the join is an **equi-join** or a **natural join**

and

an index is available on the **inner relation's join attribute**.

👉 If cost effective, can construct an **in-memory index** just to compute a join.

- For each tuple t_r in the **outer relation** r , use the index on the **inner relation** s to look up tuples in s that satisfy the join condition with tuple t_r .



Cost: $B_r + n_r * c$

- c is the cost of index search and retrieving all matching s tuples for one r tuple.
- c can be estimated as **the cost of a single selection** on s using the join condition.

If indexes are available on the join attributes of both relations,
use the smaller relation as the outer relation r .

Why?

EXAMPLE INDEXED NESTED-LOOP JOIN COST

Compute: Depositor JOIN Customer (Depositor is the outer relation)

Number of pages of $B_{\text{Depositor}} = 100$, Number of tuples $n_{\text{Depositor}} = 5000$

- Suppose Customer has a primary B⁺-tree index with 4 levels on the join attribute customerId, the primary key of Customer.

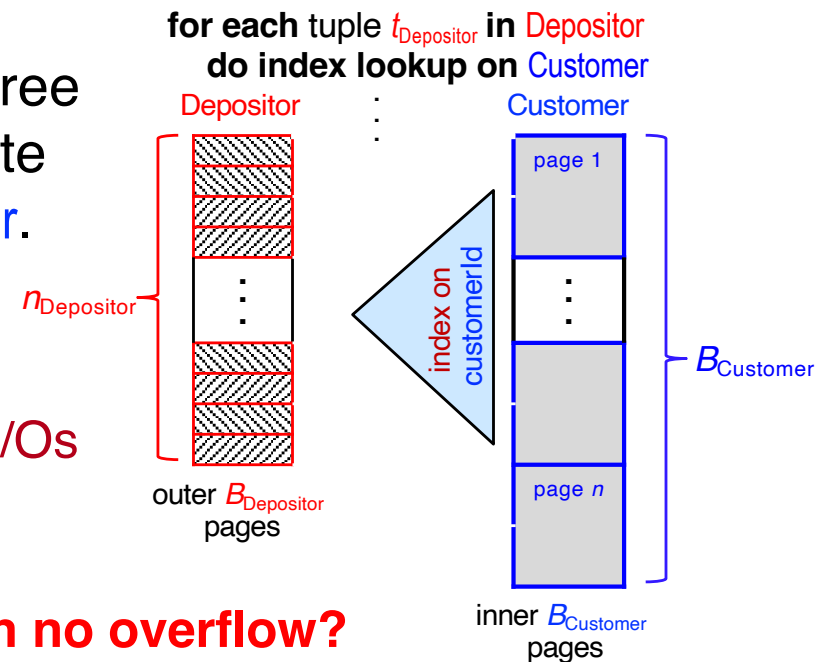
Cost: $B_r + n_r * c$

$$= B_{\text{Depositor}} + n_{\text{Depositor}} * (4 + 1)$$

$$= 100 + 5000 * 5 = \underline{25,100} \text{ page I/Os}$$

What is c ?

What is c if we have a hash index with no overflow?



Indexed nested-loop is the **best join algorithm** if there are selective conditions on the outer relation (i.e., few tuples in the outer relation satisfy the conditions).

Why?

MERGE-JOIN

- Sort both relations on their join attribute (if not already sorted on the join attributes).
- Merge the sorted relations to join them.
 - The join step is like the merge phase of the sort-merge algorithm.
 - For duplicate join attribute values
 ⇒ need to *match every pair* with the same join-attribute value.

pr	$a1$	$a2$		ps	$a1$	$a3$
→	a	3		→	a	A
	b	1			b	G
	d	8			c	L
	d	13			d	N
	f	7			m	B
	m	5				
	q	6				
			r			s

Cost: $B_r + B_s$ (if relations are sorted)
 OR
 $B_r + B_s + \text{cost of sorting}$
 (if relations are unsorted)

- Each page needs to be read in only once (assuming all tuples for any given value of the join attributes fit into the buffer).

👉 Can be used only for equi-joins and natural joins.

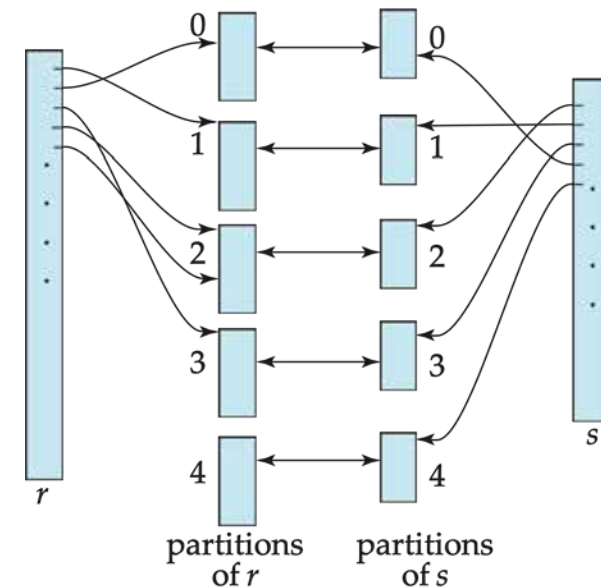
HASH-JOIN

- A hash function h is used to place tuples of both relations into n partitions (buckets) (i.e., a hash file organization).

✎ This partitions the tuples of each of the relations into sets that have the same hash value on the join attributes.

- Only need to compare r tuples in partition r_i with s tuples in partition s_i .
- Do not** need to compare r tuples in partition r_i with s tuples in any other partition, since:
 - An r and an s tuple that satisfy the join condition have the same value for the join attribute.
 - Hence, both hash to the same partition (hash value) in r and s , respectively.
 - ⇒ Each tuple must be in, respectively, partition i in r and partition i in s !

✎ Can be used only for equi-joins and natural joins.



Cost: $3 * (B_r + B_s)$

⇒ 1 read and 1 write to create the partitions;
1 read to compute the join.

HASH-JOIN ALGORITHM

/ Create partitions */*

1. Partition the relation r using the hash function h .
 - h maps *JoinAttributes* values to $\{0, 1, \dots, n-1\}$ (i.e., n partitions), where *JoinAttributes* denotes the common attributes of the natural join of r and s .
 - $r_0, r_1, \dots, r_{n-1}$ denote n partitions (buckets) of r tuples.
 - Each tuple $t_r \in r$ is in partition r_i , where $i = h(t_r[\text{JoinAttributes}])$.
 - $s_0, s_1, \dots, s_{n-1}$ denote n partitions (buckets) of s tuples.
 - Each tuple $t_s \in s$ is in partition s_i , where $i = h(t_s[\text{JoinAttributes}])$.
 - One page of the buffer is reserved as the output page for each partition.
2. Partition s similarly.

 **Need to read and write r and s to create the partitions.**

/ Perform indexed nested-loop join using hash index */*

3. For each i :
 - Load partition r_i into the buffer and build an in-memory hash index on it using the join attribute. This hash index uses a different hash function than the earlier one h . Relation r is called the **build input**.
 - Read the tuples in partition s_i from the secondary storage page-by-page. For each tuple t_s locate each matching tuple t_r in r_i using the in-memory hash index. Relation s is called the **probe input**.

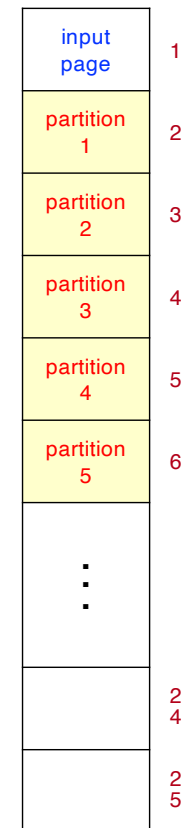
HASH-JOIN ALGORITHM (CONT'D)

- The number of partitions n is such that each partition of the build input r should fit in the available buffer pages M . Assuming each partition has the same size: $M \geq \lceil B_r/n \rceil$.
- Also, $M \geq n+1$ because for each partition we should have one buffer page (plus one page for the input buffer).
- To satisfy these conditions: $M > \sqrt{B_r}$
 - The probe relation partitions need not fit in the buffer.
- **Recursive partitioning** is required if the number of partitions n is greater than the number of pages M of the buffer.
 - **This is rarely necessary**: recursive partitioning is not needed for relations of 1GB or less with buffer size of 2MB and page size of 4KB.

EXAMPLE HASH-JOIN

Compute: Depositor JOIN Customer
 $M = 25$ pages $B_{\text{Depositor}} = 100$ $B_{\text{Customer}} = 400$

- **Depositor** is the **build input** (since it is the smaller relation).
 - Partition **Depositor** into **5** partitions, each of size **20** pages. This can be done in one pass of the **Depositor** relation (i.e., **1** read and **1** write of the **Depositor** relation).
 - Requires **6** buffer pages; **1** to read **Depositor** and **5** for the partitions.
 - Creates **5** partitions of **20** pages each. (Partitions are output pages.)
- **Customer** is the **probe input**.
 - Partition **Customer** into **5** partitions, each of size **80** pages. This is also done in one pass of the **Customer** relation (i.e., **1** read and **1** write of the **Customer** relation).
 - Requires **6** buffer pages; **1** to read **Depositor** and **5** for the partitions.
 - Creates **5** partitions of **80** pages each. (Partitions are output pages.)



✎ **The number of partitions for the two relations must be the same. Why?**

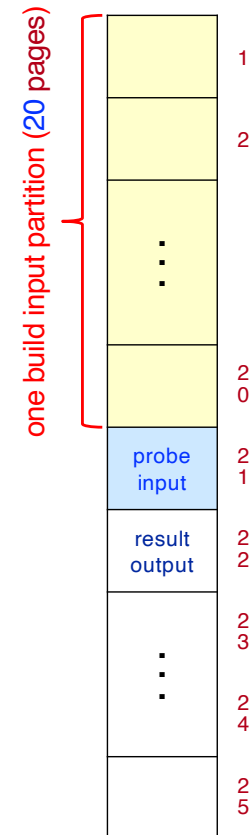
EXAMPLE HASH-JOIN (CONT'D)

Compute: Depositor **JOIN** Customer
 $M = 25$ pages $B_{\text{Depositor}} = 100$ $B_{\text{Customer}} = 400$

- In turn, read each **build input** (Depositor) **partition** (20 pages) into the buffer.
 - The build input is 5 partitions of size 20 pages each.
- ✎ **A partition of the build input must fit in the buffer. Why?**
- Read the **corresponding probe input** (Customer) partition page-by-page and probe against the **build input** partition.
 - The probe input is 5 partitions of size 80 pages each.
- Need to read both **Depositor** and **Customer** relations once to compute the join result.

Total cost: $3 * (100 + 400) = 1500$ page I/Os.

➤ This ignores the cost of writing partially filled pages.



COMPLEX JOINS

- Join with a **conjunctive condition**: $r \text{ JOIN}_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n} s$
 - Use either nested-loop/block nested-loop join, **or**
 - Compute the result of one of the **simpler joins** $r \text{ JOIN}_{\theta_i} s$
 - This join result contains those tuples that also satisfy the remaining conditions $\theta_1 \wedge \dots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \dots \wedge \theta_n$.
 - Check the remaining conditions *in the buffer*.
- Join with a **disjunctive condition**: $r \text{ JOIN}_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n} s$
 - Use either nested-loop/block nested-loop join, **or**
 - Compute the result as the **union of the records in the individual joins**

$$(r \text{ JOIN}_{\theta_1} s) \cup (r \text{ JOIN}_{\theta_2} s) \cup \dots \cup (r \text{ JOIN}_{\theta_n} s)$$

👉 **Useful only if all conditions are restrictive (selective).**

RELATIONAL ALGEBRA TREE

- Recall that a **relational algebra (operator) tree** represents a relational algebra expression (i.e., an **SQL** query) as a tree.
- It is a graphical way of representing the **evaluation order** of a relational algebra expression.
- Evaluation is **bottom up**.

👉 Can annotate the tree to indicate **access methods** to use for each relation and **implementation method** to use for each relational operator.

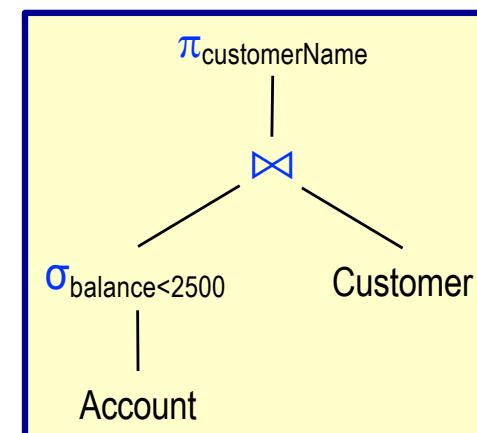
SQL query

```
select customerName
from Customer natural join Account
where balance < 2500;
```

Relational algebra expression

$\pi_{\text{customerName}}(\sigma_{\text{balance} < 2500} \text{Customer JOIN Account})$

One possible relational algebra tree

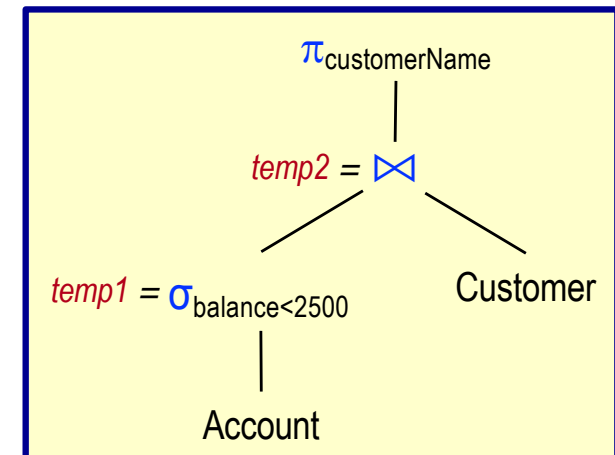


EVALUATION USING MATERIALIZATION

- Evaluate **one operation at a time**, **write operation results to a temporary relation** and use it to evaluate the next operation.

- For the relational algebra tree in the figure:

1. compute and **write to secondary storage** as **temp1** the selection $\sigma_{\text{balance} < 2500}$ Account.
2. compute and **write to secondary storage** as **temp2** the join of **temp1** with **Customer**.
3. compute the projection of **temp2** on **customerName**.

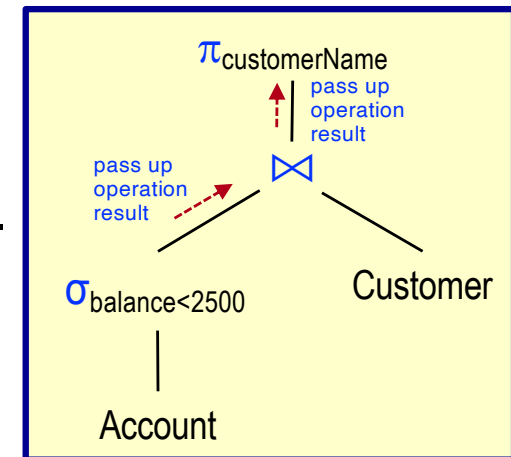


- Evaluation using materialization is **always applicable**, but the cost of writing and reading intermediate operation results can be high.

Overall cost: sum of the cost of individual operations + cost of writing operation results to secondary storage and reading them again.

EVALUATION USING PIPELINING

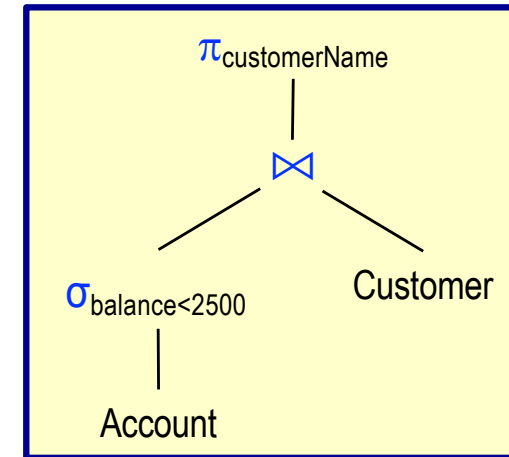
- Evaluate **several operations simultaneously**, passing the result of one operation to the next operation.
- For the relational algebra tree in the figure:
 - compute the selection $\sigma_{\text{balance} < 2500}$ Account, but **do not write** the result to secondary storage; instead, **pass the result tuples directly** to the join operation.
 - Similarly, compute the join, but **do not write** the result to secondary storage; instead, **pass the result tuples directly** to the projection operation.
- Operation evaluation can be **lazy** or **eager**.
- Pipelining is much **cheaper than materialization** since there is **no need to write operation results to secondary storage** (but may need buffer space to temporarily store each operation's result).



EVALUATION USING PIPELINING (CONTD)

Demand-driven (lazy) evaluation

- The system repeatedly requests the next tuple from the top-level operation.
- Each operation requests the next tuple from its pipelined inputs as required to output its next tuple.
- In between calls, the operation must maintain “**state**”, so it knows what to return next.



Producer-driven (eager) evaluation

- Operations produce tuples continuously and pass them into their pipeline output.
 - A buffer is needed between operations – one operation puts tuples into the buffer, while another operation removes tuples from the buffer.
 - If the buffer is full, the operation waits until there is space in the buffer before generating more tuples.

EVALUATION USING PIPELINING (CONTD)

- Algorithms that cannot output tuples as they get input tuples are called **blocking**.

✎ Merge join and hash join are blocking since they require operation results to be written to secondary storage and then read back.

- There are variants of these algorithms that generate (at least some) results on-the-fly as input tuples are read.

✎ Hybrid hash join generates output tuples even as probe relation tuples in the in-memory partition (partition 0) are read.

Pipelined Join: Hybrid hash join can be modified to buffer tuples in the first partition (partition 0) of both relations in-memory, reading them as they become available, and outputting matching result tuples between the tuples in the first partition.

- When a new r_0 tuple is found, match it with existing s_0 tuples, output matches, and save it in r_0 .
- This can also be done symmetrically for s_0 tuples.